EE2016 Microprocessors Theory and Lab, Aug - Nov 2025

Instructor: Dr. R. Manivasakan. Solutions to Tutorial 1

EE Dept, IIT Madras.

1 Fill in the blanks

1. Intel's embedded processor "embeds" the `Memory Controller Hub (MCH) and I/O Controller Hub (ICH)` into the microprocessor. Following are also the answers: "memory controllers" or "I/O controllers" or "MCH & ICH" etc
2. One can fabricate `microcontroller` by incorporating `I/O, memory` modules etc to `microprocessor`.
3. If the computational resources: super computer, wearable, main frame, desktop, server, IoT, workstation are arranged in the ascending order of their computational power, then the order would be `wearable -> IoT -> desktop -> workstation -> server -> main frame -> super computer` (Credit can be given even when the wearable and IoT are interchanged and workstation and server are interchanged).
4. The harware realization of MULT function in a microcontroller / microprocessor is a(n) `organizational` issue.
5. Data bus most of the time is `bidirectional` while the address bus is always `unidirectional`. [Exception is the data bus from program ROM is unidirectional, in Harvard architecture].
6. Major difference between the microcontroller and the microprocessor is that `the microprocessor doesn't have I/O and memory modules while the microcontroller has those modules apart from microprocessor by itself`.
7. In the Von Neumann architecture discussed in the class, the MBR handles a word of length `40 bits`. Similarly IR `8 bits`, IBR `20 bits`, MAR `12 bits`, and finally PC `12 bits`.
8. Output of IR is a part of `control signal` (address / control / data signal).
9. Issues or performance measures of whether a hardware multiplier implementation or software realization of multiplier algorithm in a microprocessor design, are `anticipated frequency of use of the multiply instruction, relative speed of the two approaches, the cost`, and `the physical size of a special multiply unit.`
10. Given a memory word of 40 bits (corresponding to a row) and 1024 such rows are there, (a) how many bits are required to uniquely identify a memory word? (b) size of memory in bits?

    `Answer:`

    (a) `10 bits`

    (b) `40960 bits or 5KB`

2 Answer the following

1. Browse in the internet for a commercial processor which has also FPGA "embedded" in it.
2. What is the difference between computer organization and architecture?
3. What is a program counter? What does it count?

4. Why is that, when the CPU frequency increases, heat dissipated also increases?

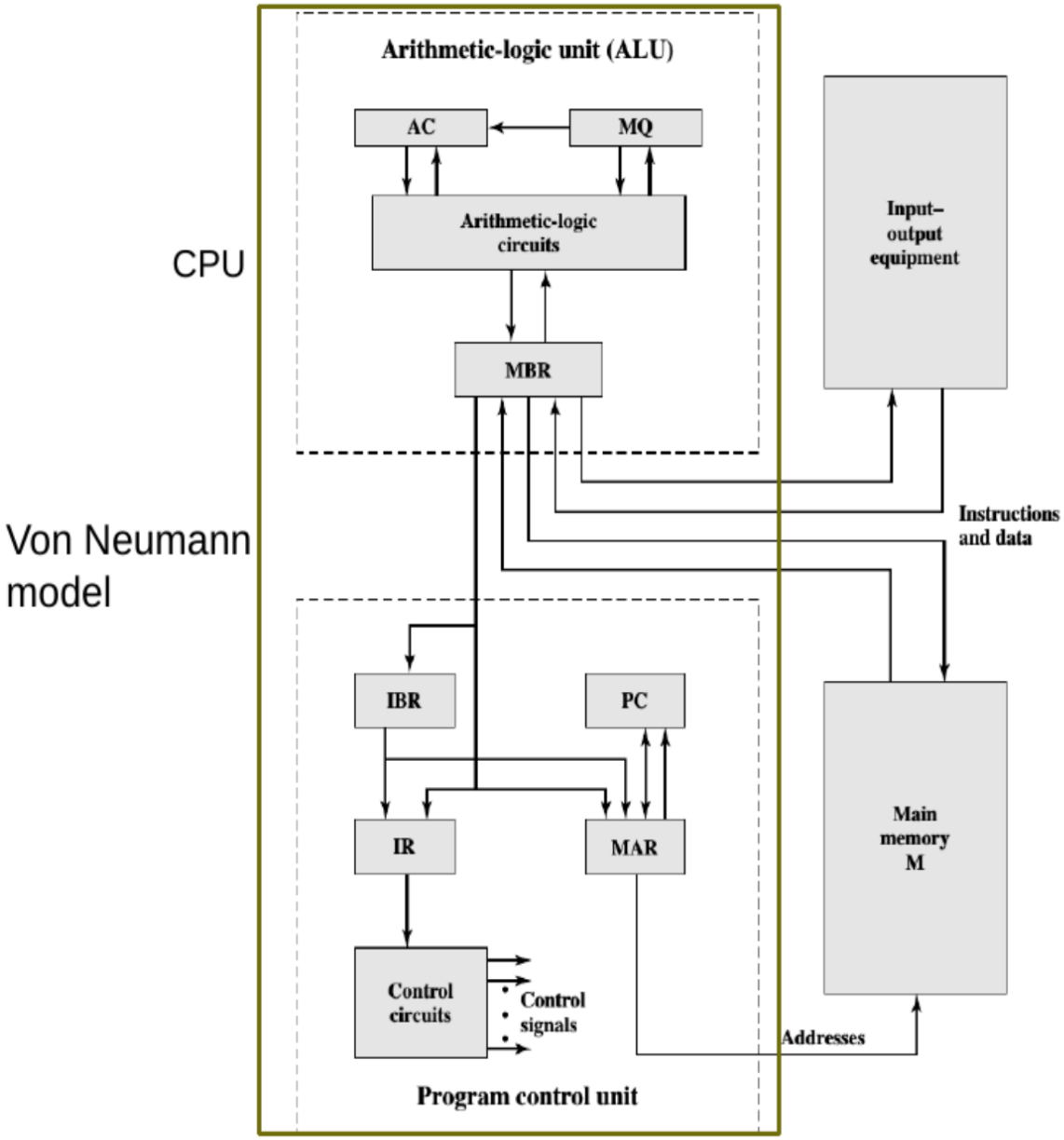
   When the CPU frequency increases, (reactive) current flows through a path corresponding to capacito (which was absent or less earlier when frequency was zero or very low). This current in real world increases Cu / resistive losses producing heat.

5. Describe the Von Neumann architecture with a block diagram and explain its operation. Indicate the bus size in each of the buses out there.

6. What is the major difference between Von Neumann architecture and Harvard architecture?

3 Solve the Following

1. Recall the van Neumann computer hardware architecture, which is given before for convenience.



   Answer the following

   (a) What is the type of bus which connects MBR from main memory? What is the size of the bus? (Meaning, how many bits does the bus possess?). What are two main types of data it can carry?

   Data bus. 40 bits. Instruction(s) and data, anyone at a time

   (b) Repeate (a) with direction changed.

   Data bus. 40 bits. Only data (from MBR to memory), it can carry.

(c) What is fed to IR block? How many bits does it have?

`OP-CODE. 8 bits.`

(d) Can MAR has something to send to PC? On the other hand, what does PC send to MAR? What is the type of bus? And its size?

`No. Address of instruction to be executed. Address bus. 12-bits`

(e) What type of bus is used to connect MAR to main memory? What is its size?

`Address bus. 12 bits.`

(f) Given that the number of rows in the main memory is 1000, and each row contains 40 bits, what is the size of memory in bytes?

`5000 bytes`

(g) Given that all the instructions are of constant length (RISC feature), of 20 bits, how many instructions are accommodated in a row in the main memory (in the program area)?

`2 instructions.`

(h) Consider the algorithmic part - flow chart - of van Neumann model. Where is that the IBR is declared 'empty', so that the condition testing block (at the top) can use that information? Indicate it in the flow chart.

`Two situations in which IBR is declared empty (a) when left instruction is NOT required (b) immediately after the right instruction in IBR is executed.`

2. Recall the van Neumann computer introduced in the class. The following Figure (Fig 1) gives the execution of instructions, in which MAR & MBR are implicit. In the following Fig, one needs to include a nibble 0 in the MSB of word corresponding to AC and IR. While the address bits are 12, the word stored in memory is also increased by 4 bits leading to 20 bit memory word. While the accumulator AC (having 20 bits in length), corresponds to MBR. 20 bits word below it, in each step, corresponds to IR (MSB 8 bits) and MAR (LSB 12 bits). For example read, "|1|9|4|0| IR" as "|0|1|9|4|0| IR MAR" wherein the LSB 12 bits (0x 940) are MAR contents and MSB 8 bits (0x 01) are IR contents. Only a generic instruction (it can be left or right instruction) is shown. AC length is also 20 bits (or 5 nibbles in length in sharp contrast with 40 bits data in van Neumann computer).

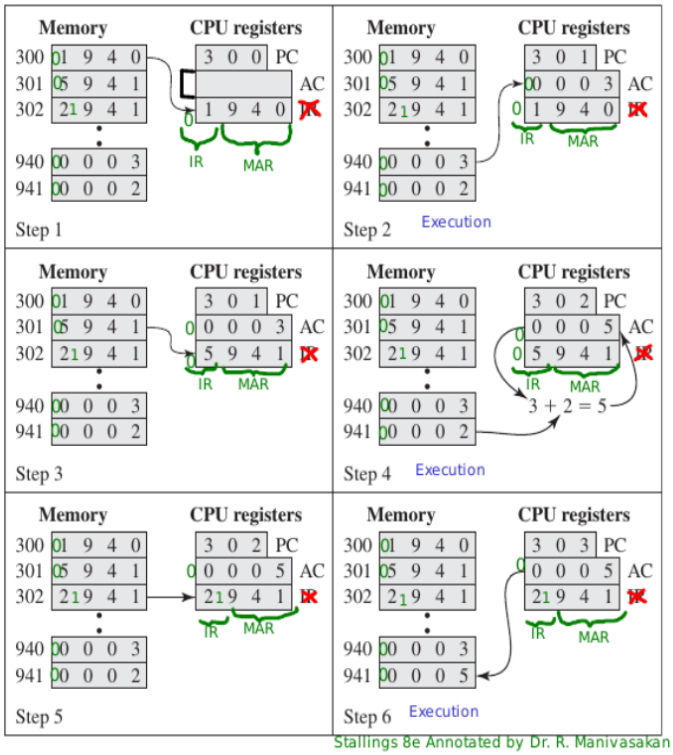


Fig 1: A program showing its execution in von Neumann's computer.

Following table gives the ISA for von Neumann first stored program computer.

Instruction Type	Opcode	Symbolic Representation	Description
Data transfer	00001010	LOAD MQ	Transfer contents of register MQ to the accumulator AC
	00001001	LOAD MQ,M(X)	Transfer contents of memory location X to MQ
	00100001	STOR M(X)	Transfer contents of accumulator to memory location X
	00000001	LOAD M(X)	Transfer M(X) to the accumulator
	00000010	LOAD −M(X)	Transfer −M(X) to the accumulator
	00000011	LOAD |M(X)|	Transfer absolute value of M(X) to the accumulator
	00000100	LOAD −|M(X)|	Transfer −|M(X)| to the accumulator
Unconditional branch	00001101	JUMP M(X,0:19)	Take next instruction from left half of M(X)
	00001110	JUMP M(X,20:39)	Take next instruction from right half of M(X)
Conditional branch	00001111	JUMP+ M(X,0:19)	If number in the accumulator is nonnegative, take next instruction from left half of M(X)
	00010000	JUMP+ M(X,20:39)	If number in the accumulator is nonnegative, take next instruction from right half of M(X)
Arithmetic	00000101	ADD M(X)	Add M(X) to AC; put the result in AC
	00000111	ADD |M(X)|	Add |M(X)| to AC; put the result in AC
	00000110	SUB M(X)	Subtract M(X) from AC; put the result in AC
	00001000	SUB |M(X)|	Subtract |M(X)| from AC; put the remainder in AC
	00001011	MUL M(X)	Multiply M(X) by MQ; put most significant bits of result in AC, put least significant bits in MQ
	00001100	DIV M(X)	Divide AC by M(X); put the quotient in MQ and the remainder in AC
	00010100	LSH	Multiply accumulator by 2; i.e., shift left one bit position
	00010101	RSH	Divide accumulator by 2; i.e., shift right one position
Address modify	00010010	STOR M(X,8:19)	Replace left address field at M(X) by 12 rightmost bits of AC
	00010011	STOR M(X,28:39)	Replace right address field at M(X) by 12 rightmost bits of AC

(a) List the instructions used in program shown in Fig 1 above (All binary representations are in HEX).

i. Instruction in step 1 & 2 is |0|1|9|4|0| in which the OP-Code is 01 and address for operand is 940 (12 bits). OP-CODE HEX 01 in the above table tells that this corresponds to ‘‘LOAD’’ the accumulator with the contents @ address 940.

ii. Instruction in step 3 & 4 is |0|5|9|4|1| in which the OP-Code is 05 and the address for operand is 941 (12 bits). OP-CODE HEX 05 in the above table, means this code ADDs contents of AC to that given location (here it is @ 941) and puts the result back in AC.

iii. Instruction in step 5 & 6 is |0|2|9|4|1| in which the OP-Code is 21 and the address for operand is 941 (12 bits). From the above table, the OP-CODE HEX 21 means, STORE the AC contents to a given location. Here the given location is HEX 941.

Note that the 1st part of each row is “fetching” and 2nd part in the same row is “executing”.

(b) Quantitatively evaluate the value of MAR in each clock cycle (steps 1 to 6 as shown above).

i. Content of MAR at the end of steps 1 & 2 is 940

ii. Content of MAR at the end of steps 3 & 4 is 941

iii. Content of MAR at the end of steps 5 & 6 is 941

(c) What is the role of AC here?

Role is to hold the operands value temporarily, do additions (or subtractions) of another incoming data & holding it, till the final result is sent to main memory. This is called ‘‘accumulator’’ since the results gets accumulated (added in sequence) in many practical computat environments.

3. The stray capacitance, frequency and supply voltage, in an intel microprocessor chip and an AVR microcontroller chip, respectively are given as below:

   (a) intel processor chip 10 nF, 1.72 V and frequency 6 GHz power.
   (b) AVR microcontroller chip 5 V, 100 pF operating at 1 MHz (0.25 W).

   Given the above, answer the following:

   (a) Compute the heat (dynamic power) generated (which is to be removed or dissipated by cooling mechanism)
   (b) What is the heat removing rate (in J/sec) to be implemented by the cooling mechanism for each of the above, so that temperature of the chip doesnt increase?
   (c) While, any heat generated, less than a watt could be ignored (meaning, the heat generated, would either be naturally be dissipated or would not raise the temperature of the chip substantially), which of the above (intel processor or AVR microcontroller) doesn't need any cooling mechanism.

   Solution:

   (a) Dynamic power dissipated
      i. power dissipated in intel processor = $CV^2f = 177.5\ W.$
      ii. Power dissipated in AVR microcontroller chip = $CV^2f = 100 \times 10^{-6} \times 25 = 0.0025\ W = 2.5\ mW$
   (b) Heat removing rate
      i. for intel processor 177.5 Jouls/sec
      ii. for AVR microcontroller, 2.5 mJoules/sec

      Depending on the chip physics (including fabrication chemistry), some minimum amount of heat transfer happens naturally, say 1 Joules / sec. Hence, we don't need any special heat removing mechanisms for the AVR microcontrollers, while for intel processors do need.
   (c) AVR microcontroller.

4. Consider a 64-bit intel processor having a 32-bit address bus, with 4 control lines (corresponding to RD/WR, chip enable etc) is proposed to be connected to the main memory (housing program and data) through (a) van Neumann architecture (b) Harvard architecture. Compute the total number of wire-traces (system bus width) to be used in each of the above cases. Give the corresponding picture. [Note that this problem concerns only the part of the system bus which connects to memory, not the part connecting to I/O].

   Solution:

   (a) von Neumann architecture: 64+32+4 = 100 wire traces
   (b) Harvard architecture: 64+32+4 (Program ROM) + 64+32+4 (SRAM for data)=200 wire traces

5. **van Neumann versus Harvard Architecture - Computational Time:** To see the advantage of Harvard architecture, consider an assembly program of an engineering task which is implemented in AVR processor (operating at 200 MHz) and intel processor (operating at 2 GHz). The RISC program in AVR contains 100 lines (each has external SRAM access of 1 clock cycle for operand, execution & writeout - concurrently ROM fetch of next instruction and so on). The CISC program in intel takes 50 lines, in which given an instruction following number of clock cycles are consumed: first 10 cycles are for instruction fetch, next 10 cycles are data (operand) fetch and next 20 cycles are for execution. The last 10 cycles are writeout (WO). Next instruction can be fetched during execution of previous instruction (primitive pipelining). Compute the time taken to execute for the whole program by the AVR and intel processor.

   Solution:

   AVR - Harvard Architecture (& RISC Architecture) The AVR uses a Harvard architecture, which allows for simultaneous fetching of the next instruction while the current one is being executed.

   (a) Program Size: 100 lines
   (b) Operating Frequency: 200 MHz Clock period: 1/(200×10 ^6) Hz = 5×10^ −9 sec = 5 nsec
   (c) Since AVR takes one clock cycle per instruction (as instruction fetch, execution, and write-out happen concurrently), the total time is given by 100 instructions × 1 clock cycle/instruction × 5 ns/clock cycle = 500 ns

intel processor - van Neumann Architecture (& RISC Architecture) Assumes that intel processor uses primitive pipeline (in realworld, intel uses much more powerful, sophisticated pipelining and superscalar architecture). This primitive pipeline allows for fetching the next instruction while the previous one is in its execution stage - only assumption for the problem.

Number of instructions - 50. Clock period = $\frac{1}{2\times10^9} = 0.5 \times 10^{-9} = 0.5$ nsec. Total time to execute the above program (incorporating the pipelining, meaning while next instruction fetch (10 clock periods) and data fetch (10 clock periods) happen in series in time axis, the previous instruction's execution (20 clock periods) happen concurrently. But, 3rd instruction can only pipeline the instr fetch with 2nd part of Exe cycle of previous instruction in next slot WO of previous instruction prohibits data fetch. This means 3rd instruction takes independently 40 clock period. Fourth instr can overlap to have only 20 independent clock period. This pattern repeats. And, hence total time = time for 1st instr + (49 instr x 30 clock period)= $50 + 1470$ cycles $= 1520 \times 0.5$ ns/clock period = 760 nsec. [30 clock periods is the average of 20 & 40].

Despite the (mighty) Intel processor having a higher clock speed and fewer lines of code, the AVR's more efficient Harvard architecture (instruction & data fetch can be concurrent) and single-cycle instruction execution gives, it a 50 % advantage in latency, in this specific example.